Solving Fredholm Integral Equations by Writing Code:
A Program-of-Thought Experiment on `test_100_v2`

Fred-LLM project

13 July 2026

Abstract

Frontier models are better programmers than they are algebraists, so we stop asking them to do algebra. Each equation in the benchmark is re-rendered as a short block of executable SymPy definitions, and the model is asked for a Python script that solves it. We run that script in a sandbox with a 60-second timeout and one repair round, and score whatever it prints with the repository's unchanged evaluation pipeline. On the 100-equation `test_100_v2` set this reaches 42/100 with gpt-5.4, the best result we have from any GPT-family method: the plain-prompt baseline scores 35 and the five-agent agentic workflow scores 38 at 2.3 times the cost. On gpt-5.4-mini the method scores 35/100, which matches the plain gpt-5.4 baseline at about a quarter of its price. An ablation that shows the model the same code-shaped prompt but takes its answer directly, without execution, gains only 3 points over the baseline; execution is what pays, not the representation. The method has one systematic failure: it finds almost none of the 15 no-solution items. Most scripts do attempt a singularity check, but they choose tolerances that a float system at these magnitudes never trips. The fine-tuned 360M specialist from the previous report keeps the overall lead (46 plain, 53 with solver routing), and it wins exactly where the scripts lose, on the decision items.

1 Motivation

The July 7 agentic pilot bought its accuracy with orchestration: five method agents per equation, a SymPy verifier, and a repair round, at $10.62 per hundred equations on gpt-5.4. The July 12 fine-tuning report then showed that a deterministic degenerate-kernel solver, called at the right moments, solves more of this benchmark than any GPT prompt. Both results point the same way. The hard part of a separable-kernel equation is not choosing the method; it is carrying out pages of exact arithmetic on coefficients like -797089.486. Our working hypothesis was that models can set this arithmetic up but cannot carry it out.

Program-of-thought prompting (PAL, PoT) splits the two: the model writes the setup as code, an interpreter does the arithmetic. This experiment tests that split on Fredholm equations, and uses an ablation to check where any gain comes from.

2 Method

2.1 Code representation

`experiments/code_prompt/build_prompts.py` converts each `test_100_v2` item (kernel, right-hand side, λ, domain, all taken from the same metadata the basic prompts use) into executable definitions:

```
from sympy import *
x, t = symbols('x t')
lam = 0.6800510922329444
a, b = -1.1508025669664157, 9.746297944109074
K = sin(x)*cos(t)      # kernel K(x, t)
f = ...                # right-hand side f(x)
# u(x) - lam * Integral(K * u(t), (t, a, b)) = f
```

LaTeX-to-SymPy conversion goes through the repository's `parse_latex_to_sympy`; all 100 items convert, and every generated block was executed locally as a sanity check before any API call. Items with $\lambda = 0$ are rendered as first-kind equations, the same rule the basic renderer applies, so the prompts leak no ground truth.

2.2 Two variants

The `code` variant shows the model this block and asks for an answer directly, in the standard `SOLUTION:` format. It isolates the effect of seeing the equation as code rather than as LaTeX.

The `code_exec` variant asks for one self-contained Python script that solves the equation and prints the same three-line format, with the solution in LaTeX via `sympy.latex()`. A new wrapper, `CodeExecModelRunner` (`src/llm/code_exec_runner.py`, enabled by a `model.code_exec` config section), extracts the script, runs it in an isolated subprocess (`python -I`, 60-second timeout), and feeds one failure back to the model for a single repair round. If the script still fails, the raw model text is scored instead. The script's stdout replaces the model's response, so the existing parser, evaluator, and metrics apply without modification. This is the same wrapping pattern the agentic runner uses, which keeps every number in this report comparable to the earlier ones.

Both prompts state the degenerate-kernel recipe (reduce to a linear system for the integral coefficients; singular and consistent means a family, singular and inconsistent means no solution). The agentic workflow's method directives say the same thing, so no method receives hints the others lack.

2.3 Runs

Temperature 0.1, `max_tokens` 4096, and the evaluation tolerances of the July 7 pilot (series 0.01, approximate-coefficient 0.001, regularized 0.001), all through the same router. An 8-item pilot ran first, one item per solution type plus one extra, at $0.11 for both variants together. The full runs cost $0.76 (mini, `code`), $0.78 (mini, `code_exec`), and $4.71 (gpt-5.4, `code_exec`). We skipped the gpt-5.4 `code` run: the mini ablation had already answered the representation question, and the extra $3 would not have changed any conclusion.

3 Results

`code_exec` beats the basic prompt by 14 points on mini and 7 on gpt-5.4, and beats the agentic workflow on both models while using one model call per equation instead of five to ten. The representation-only ablation gains 3 points. Reading the equation as code barely matters; running the model's code is the mechanism.

The exact-symbolic column carries the result. The basic mini prompt solves 6 of 30; the same model writing scripts solves 20 of 28. gpt-5.4 reaches 23 of 29, and 6 of its 10 first-kind items,

Model	Method	Correct/100	Type acc.	Cost
gpt-5.4-mini	basic prompt	21	0.253	—
gpt-5.4-mini	agentic multi-method	32	0.310	—
gpt-5.4-mini	`code` (no execution)	24	0.256	$0.76
gpt-5.4-mini	`code_exec`	**35**	0.198	$0.78
gpt-5.4	basic prompt	35	0.385	$3.01
gpt-5.4	agentic multi-method	38	0.420	$10.62
gpt-5.4	`code_exec`	**42**	0.244	$4.71
SmolLM2-360M	plain SFT	46	0.680	local
SmolLM2-360M	plain SFT + solver routing	**53**	0.680	local
(no LM)	separable solver always	28	0.300	free

Table 1: `test_100_v2` results. Baseline, agentic, and SmolLM2 rows are from the July 7 and July 12 reports. "Correct/100" counts unparseable responses as wrong.

Method	exact	approx	series	discrete	family	regul.	none
mini, basic	6/30	3/14	0/10	0/9	10/10	0/10	2/15
mini, agentic	13/30	2/15	0/10	0/10	10/10	2/10	5/15
mini, `code`	9/30	0/15	0/9	0/9	10/10	0/10	5/15
mini, `code_exec`	**20/28**	3/13	0/10	0/7	10/10	2/10	0/15
gpt-5.4, `code_exec`	**23/29**	2/14	0/9	0/8	10/10	**6/10**	1/15

Table 2: Per-type correct counts. Denominators below the type's size mean some responses for that type failed to parse.

the best regularized score of any GPT method. The scripts also keep the family items at 10/10 because the models special-case $f = 0$ in code and report the resonant family.

Script reliability was high on both models. On mini, 92 of 100 scripts ran on the first try and one response was prose with no code block; the repair round fixed 5 of the remaining 7, and the two that stayed broken were a 60-second timeout and a persistent traceback. On gpt-5.4, 87 ran first try and the repair round fixed 9 of 13, leaving 4 items scored on raw text. Repairs are the difference between the 100 prompts and the 107 and 113 requests the cost tracker recorded.

4 Why execution wins, and where it fails

Solving $u(x) = f(x) + \lambda \sum_i c_i g_i(x)$ requires integrating products of the h_i against f and inverting a small matrix with coefficients that run to six significant figures times 10^5. The baseline responses at temperature 0.1 return only a final answer, so we cannot see where the internal derivation goes wrong; what the results show is that the same model that gets 6 (mini) or 13 (agentic mini) of the exact items right on its own gets 20 right once an interpreter does the arithmetic. The generated scripts are not trivial, the median is 70 lines, but every line is setup: define the kernel factors, build the coefficient matrix, hand it to SymPy. The ability being measured shifts from symbolic manipulation to problem setup, and setup saturates earlier. Mini plus an interpreter equals gpt-5.4 alone, at 26% of its cost.

The failure is just as mechanical, and it is not for lack of trying. Every no-solution item in this benchmark is a resonance case: λ sits at an eigenvalue, the linear system is singular, and the inhomogeneous part is inconsistent with it. 73 of the 100 gpt-5.4 scripts, and 50 of the mini ones, do attempt a rank, determinant, or conditioning check. The checks fail anyway: one

representative gpt-5.4 script computes at 80-digit precision and then tests `rref` pivots against a 10^{-28} threshold, which no near-singular system with coefficients of order 10^5 will ever cross. The float system looks regular, `linsolve` returns a huge answer, and the script prints it with `HAS_SOLUTION: yes`. Mini lost all 15 such items, gpt-5.4 lost 14. Choosing a numerically meaningful tolerance turns out to be the one part of the setup the models cannot do either. Type accuracy drops for a related reason: a script that computed a float answer tends to label it `exact_symbolic` regardless of what the item actually is.

This is why the fine-tuned specialist still leads. The evaluator scores 35 of the 100 items purely on decisions (no-solution, family, regularized), and decisions are what the SFT model learned and what scripts are worst at. The two failure profiles barely overlap, which suggests a cheap hybrid: let something else own the no-solution decision (the SFT router, or a conditioning check whose tolerance the harness fixes instead of the model), and let the script own the algebra. The solver-routing result from the previous report (46 to 53 from routing alone) is the same shape of fix.

5 Comparison with the agentic workflow

Agentic buys its lift with parallel method agents, a residual verifier, and 2.2 times the requests of a baseline run. `code_exec` spends one call plus a subprocess and gains more on both models. The two are orthogonal rather than competing: the agentic verifier could score script outputs, and a script is in effect a fifth method agent whose work is checked by an interpreter instead of a judge. The scaffolding-helps-weaker-models pattern from the agentic pilot repeats here, 14 points of lift on mini against 7 on gpt-5.4, at roughly triple the magnitude the agentic workflow managed.

6 Limitations

Single run per configuration at temperature 0.1, so the numbers carry sampling noise we have not measured; the 3-point representation effect in particular is within that noise. The first-kind items are named as first-kind in a code comment, which is one step more explicit than the basic prompt's rendering (the form is visible in both, but the baseline model has to recognize it); part of the 6/10 regularized score may be that labeling. Series and discrete-point items score zero under every GPT method, including this one, and scripts do not change that: those items need a different output contract, not better arithmetic. The sandbox is a subprocess with a timeout, not a hardened jail; that is fine for SymPy scripts from a model solving math and would not be fine for arbitrary code. And the killed first attempt at the gpt-5.4 run cost an unrecoverable partial batch, since the `code_exec` path has no checkpointing; long runs should add the per-equation checkpoint the agentic runner already has.

7 Reproducing

```
.venv/bin/python experiments/code_prompt/build_prompts.py
.venv/bin/python -m src.cli run --config configs/test_100v2_gpt54_code_exec.yaml
.venv/bin/python experiments/code_prompt/compare.py
```

Artifacts, including every generated script and its execution record, are in `results/code_prompt_2026-07-13/`. The narrative version of this report is `experiments/code_prompt/REPORT.md`.